

## **Taller 2: Redes Neuronales**



### **Integrantes:**

Daniela Martinez Cañaveral

Carlos Arturo Triana

### **Docente**

Fredy Alexander Orjuela López

**Universidad ECCI**

**Facultad de Ingenierías, Programa de Ingeniería Industrial**

**Sistemas Avanzados de Producción**

## Fase 1: Desarrollo Analítico: Regularización Ridge.

- Matricial

$$J(\beta) = (Y - X\beta)^T(Y - X\beta) + \lambda\beta^T\beta$$

- Expansión

$$(Y - X\beta)^T(Y - X\beta) = Y^TY - Y^TX\beta - \beta^TX^TY + \beta^TX^TX\beta$$

- Simplificación

$$J(\beta) = Y^TY - 2\beta^TX^TY + \beta^TX^TX\beta + \lambda\beta^T\beta$$

- Agrupación

$$J(\beta) = Y^TY - 2\beta^TX^TY + \beta^T(X^TX + \lambda I)\beta$$

- Derivada

$$\frac{\partial J(\beta)}{\partial \beta} = -2X^TY + 2(X^TX + \lambda I)\beta$$

- Igualar a cero

$$-2X^TY + 2(X^TX + \lambda I)\beta = 0$$

- Despeje

$$(X^TX + \lambda I)\beta = X^TY$$

$$\beta_{\text{ridge}} = (X^TX + \lambda I)^{-1}X^TY$$

- **Sobre la invertibilidad:** ¿Porque la suma de  $\lambda I$  garantiza que la matriz resultante sea invertible incluso si  $X^T X$  es singular (no invertible)?

Una matriz no es invertible cuando presenta columnas duplicadas o información redundante (es decir, es singular). Al agregar  $\lambda$ , se introduce un valor pequeño en la diagonal principal. Esto elimina cualquier dependencia exacta entre los datos, garantizando que la matriz conserve “volumen” y que su inversa pueda calcularse siempre, independientemente del nivel de correlación entre las variables originales.

- **Efecto del Parámetro  $\lambda$ :** ¿Que sucede con el modelo y sus coeficientes cuando  $\lambda \rightarrow 0$ ? ¿Y qué ocurre cuando  $\lambda \rightarrow \infty$ ? Explique en términos de la complejidad del modelo (Sesgo vs. Varianza).

Cuando  $\lambda \rightarrow 0$ : El modelo es una regresión lineal común. Se esfuerza tanto por pasar por cada punto que se vuelve muy complejo y tiene bajo sesgo, pero alta varianza.

Cuando  $\lambda \rightarrow \infty$ : La penalización es tan fuerte que aplasta los coeficientes, obligándolos a ser casi cero por lo cual el modelo se vuelve extremadamente simple teniendo baja varianza, pero alto sesgo.

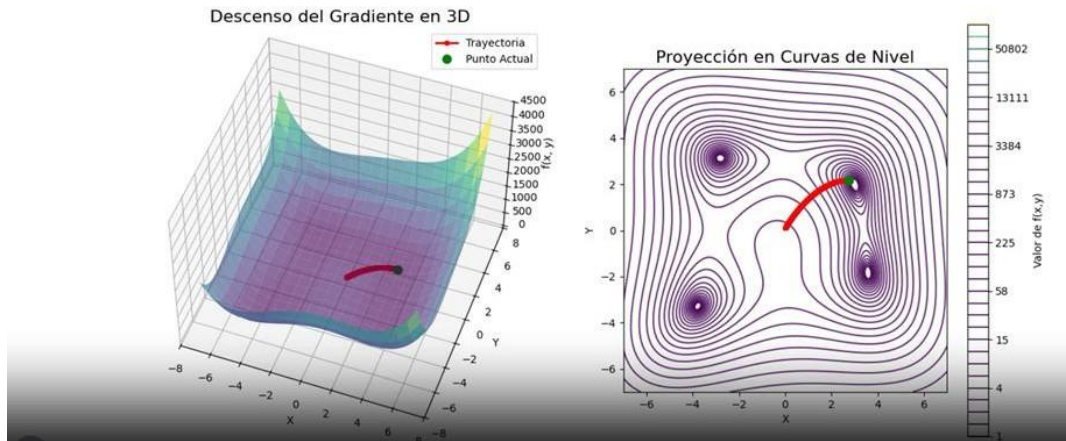
## Fase 2: Optimización Numérica y Descenso de Gradiente.

### 2.1. Impacto de la Tasa de Aprendizaje ( $\eta$ )

- Mantenga el start point y las numeraciones constantes.

Corriendo el código sin modificaciones, inicialmente la malla 3D sale así.

```
# --- CONFIGURACIÓN DE LA SIMULACIÓN ---  
# Punto de inicio (puedes cambiarlo para ver cómo cae en otro mínimo)  
start_point = [0.0, 0.1]  
learning_rate = 0.0001      # Tasa de aprendizaje  
num_iterations = 1000       # Número de pasos
```



- **Incremente el learning rate a un valor significativamente más alto (ej. 0.5 o 1.0).** ¿Qué sucede con la trayectoria? ¿Logra llegar al centro de las curvas de nivel o comienza a oscilar/divergir?

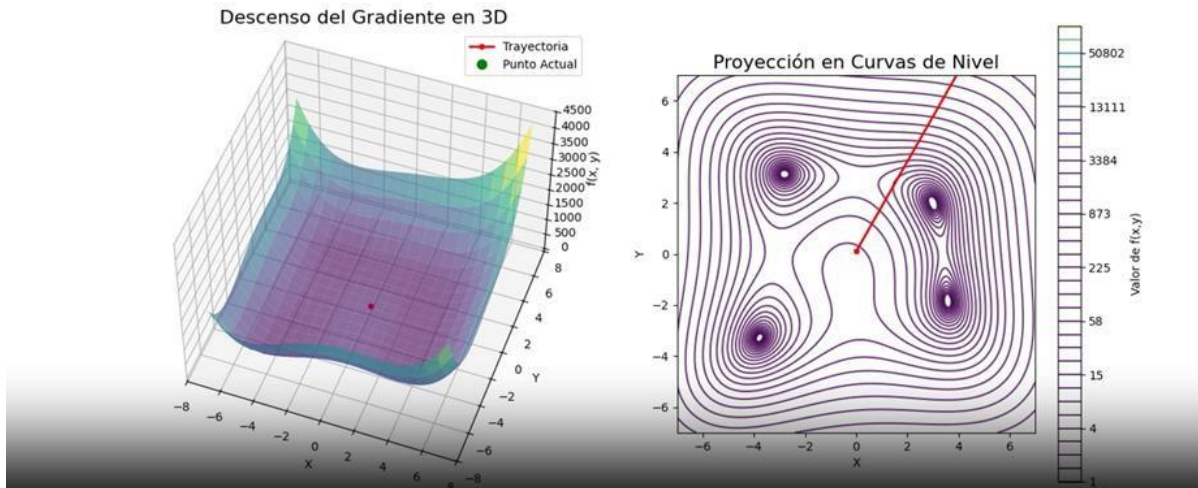
En el análisis matemático y computacional, especialmente en contextos como la estadística, el aprendizaje automático y el álgebra lineal aplicada, la invertibilidad de una matriz es una propiedad fundamental. Una matriz es invertible únicamente si es no singular, lo que implica que no presenta redundancia en su información. En términos prácticos, esto significa que ninguna de sus columnas (o filas) puede expresarse como combinación lineal de las demás. Cuando existen columnas repetidas o altamente correlacionadas, la matriz se vuelve singular o casi singular, lo cual impide calcular su inversa de manera estable o incluso hacerlo en absoluto.

Este problema es común en situaciones donde las variables de entrada están fuertemente correlacionadas, fenómeno conocido como multicolinealidad. En estos casos, la matriz asociada (por ejemplo, la matriz de covarianza o la matriz  $X^T X$  en regresión lineal) puede volverse mal condicionada, lo que genera inestabilidad numérica y resultados poco fiables al intentar invertirla..

Además, al incrementar la tasa de aprendizaje a 1.0, se observa que el algoritmo no logra converger al mínimo, la trayectoria muestra un comportamiento de divergencia, donde los pasos son tan grandes que superan la zona de los valles y se alejan del centro de las curvas de nivel, incrementando el valor de la función de pérdida en cada paso, esto explica que cuando el paso es muy grande, el algoritmo "se pasa" del punto más bajo y termina subiendo por la pared opuesta del valle como se ve en la imagen una trayectoria que se aleja del centro.



```
# --- CONFIGURACIÓN DE LA SIMULACIÓN ---
# Punto de inicio (puedes cambiarlo para ver cómo cae en otro mínimo)
start_point = [0.0, 0.1]
learning_rate = 1.0      # Tasa de aprendizaje
num_iterations = 1000     # Número de pasos
```



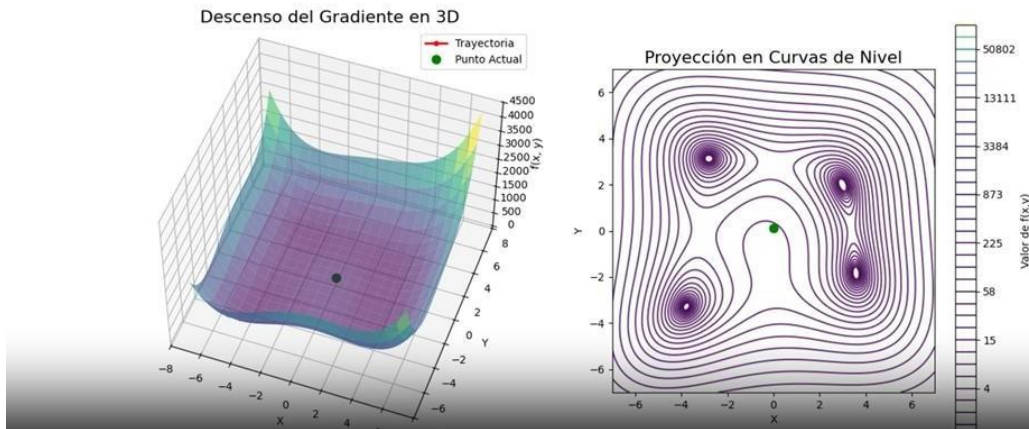
- **Reduce el aprendizaje a un valor extremadamente pequeño ej.  $10^{-7}$  Observe el resultado tras 1000 iteraciones.** ¿Es suficiente este tiempo de cómputo para alcanzar el mínimo?

En la gráfica el punto verde parece estar completamente estático a diferencia del experimento anterior donde la línea roja se disparaba hacia afuera, en este caso la trayectoria es prácticamente invisible porque no ha logrado avanzar.

Al utilizar una tasa de aprendizaje de 0.0000001, el algoritmo está dando pasos muy microscópicos, aunque el gradiente le indica la dirección correcta hacia el mínimo, la magnitud del avance en cada una de las 1000 iteraciones es tan pequeña que el algoritmo queda atrapado casi en el mismo lugar donde empezó, permaneciendo prácticamente en la posición inicial [0.0, 0.1], lo que indica que el tiempo de cómputo no es suficiente.

Este fenómeno demuestra que una tasa demasiado baja hace que la convergencia sea inviable, requiriendo un ajuste hacia arriba para lograr un avance significativo hacia el mínimo.

```
# --- CONFIGURACIÓN DE LA SIMULACIÓN ---
# Punto de inicio (puedes cambiarlo para ver cómo cae en otro mínimo)
start_point = [0.0, 0.1]
learning_rate = 0.0000001      # Tasa de aprendizaje
num_iterations = 1000          # Número de pasos
```

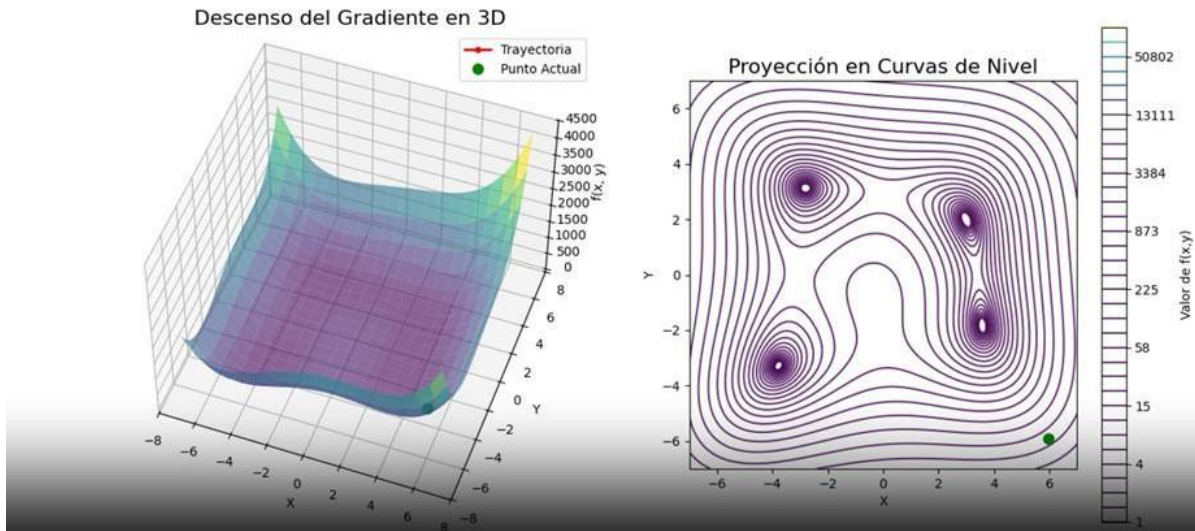


## 2.2. Dependencia del Punto Inicial.

- Modifique el start point a coordenadas alejadas del origen, por ejemplo [6.0, -6.0] o [-5.0, 5.0].

Modificación del start point a las coordenadas 6.0, -6.0.

```
# --- CONFIGURACIÓN DE LA SIMULACIÓN ---
# Punto de inicio (puedes cambiarlo para ver cómo cae en otro mínimo)
start_point = [6.0, -6.0]
learning_rate = 0.0000001      # Tasa de aprendizaje
num_iterations = 1000          # Número de pasos
```



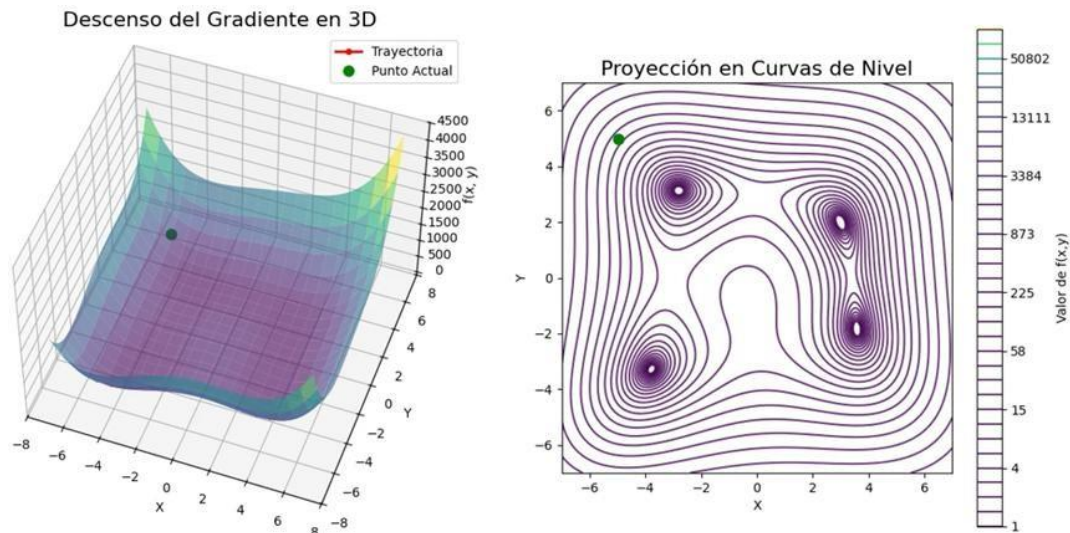
### Modificación del start poitn a las coordenadas -5.0, 5.0

```
# --- CONFIGURACIÓN DE LA SIMULACIÓN ---
# Punto de inicio (puedes cambiarlo para ver cómo cae en otro mínimo)
start_point = [-5.0, 5.0]
learning_rate = 0.000001      # Tasa de aprendizaje
num_iterations = 1000         # Número de pasos
```

Al modificar el punto inicial (Start point), se observa que el algoritmo no llega siempre al mismo punto final por que la trayectoria y el destino dependen enteramente de donde comienza la búsqueda.

La topografía de la función  $f(X,Y)$  sugiere una superficie no convexa con múltiples valles. Esto confirma la presencia de mínimos locales, donde el gradiente se vuelve cero y atrapa al algoritmo, impidiéndole explorar otros valles que podrían contener el mínimo global.

Esto significa que en problemas reales, no sabemos dónde está el hoyo más profundo si solo empezamos en un punto, podríamos quedar atrapados en una solución subóptima.



## Preguntas Fase 2

- El fenómeno de “Overshooting”: Explique físicamente qué ocurre cuando el paso ( $\eta \cdot \nabla f$ ) es más grande que la distancia al mínimo. ¿Cómo se visualiza esto en el gráfico de curvas de nivel?

Ocurre cuando la magnitud de la actualización es tan grande que el algoritmo "salta" por encima del punto mínimo.

Físicamente, es como si una partícula con demasiada energía cinética intentara detenerse en el fondo de un pozo, pero termina aterrizando en una zona de mayor energía (altura) en la pared opuesta.

En el gráfico, esto se observa como una trayectoria que cruza el centro de forma errática o que rebota violentamente entre curvas de nivel externas en donde la línea roja se disparaba hacia afuera, alejándose de los valles; esto es un "overshooting" extremo que lleva a la divergencia.

- **Eficiencia Computacional:** Si el algoritmo se detiene en la iteración 1000 pero aún se encuentra lejos del mínimo, ¿qué estrategia es mejor: aumentar el número de iteraciones o ajustar la tasa de aprendizaje? Justifique su respuesta.



En la prueba con  $10^{-7}$ , se evidencia que tras 1000 iteraciones el punto apenas se movió, si simplemente se aumentan las iteraciones a 1.000.000 se estaría gastando una cantidad masiva de tiempo y recursos de cómputo para lograr un avance que una tasa de aprendizaje bien sintonizada 0.01 se lograría en solo 50 pasos permitiendo que el algoritmo sea eficiente, llegando a la solución con el menor gasto computacional posible, por lo cual; la mejor estrategia es ajustar (aumentar) la tasa de aprendizaje en lugar de aumentar las iteraciones.

- **Generalización: En un problema de ingeniería con cientos de variables (como la optimización de una cadena de suministro), ¿por qué es peligroso confiar en un único punto de inicio aleatorio?**

En problemas complejos con muchas variables, la superficie de optimización no es convexa como se evidencia en la gráfica de curvas de nivel, que tiene al menos 4 valles distintos, se podría quedar atrapado en una solución que parece buena pero que es muy inferior al mínimo global (la solución óptima real).

En una cadena de suministro, esto podría significar costos innecesariamente altos o rutas ineficientes simplemente por no haber explorado el terreno desde diferentes puntos de partida.

### **Fase 3: Redes Neuronales (MLP) y Ajuste de Hiperparametros**

#### **Preguntas Fase 3A:**

- **Diferenciación de Métricas:** Explique la diferencia técnica entre el Error Absoluto Medio (MAE) y la Raíz del Error Cuadrático Medio (RMSE).

**MAE (Error Absoluto Medio):** Calcula el promedio de las diferencias absolutas entre los valores predichos y los valores reales. Se expresa en las mismas unidades que la variable objetivo. Su fórmula es  $MAE = (1/n) * \sum |y_{real} - y_{pred}|$ .

**RMSE (Raíz del Error Cuadrático Medio):** Calcula la raíz cuadrada del promedio de los errores al cuadrado. También se expresa en las mismas unidades que la variable objetivo. Su fórmula es  $RMSE = \sqrt{(1/n) * \sum (y_{real} - y_{pred})^2}$ .

**¿Cuál de las dos penaliza con mayor severidad los errores grandes (outliers) en las ventas y por qué?**

El RMSE penaliza con mayor severidad los errores grandes (outliers). Esto se debe a que, en su cálculo, los errores se elevan al cuadrado. Al elevar al cuadrado, los errores grandes se magnifican mucho más que los errores pequeños. Por ejemplo, un error de 10 se convierte en 100, mientras que un error de 1 se convierte en 1. Aunque al final se saca la raíz cuadrada, el efecto de amplificación de los errores grandes continua, haciéndolo más sensible a los errores en comparación con el MAE, que simplemente toma el valor absoluto de los errores, tratándolos de manera lineal.

• **Diagnóstico de Desempeño:** Si el R2 en el conjunto de entrenamiento es significativamente mayor que en el de prueba, ¿qué fenómeno está ocurriendo?:

Si el R2 en el conjunto de entrenamiento es significativamente mayor que en el de prueba, el fenómeno que está ocurriendo es el sobreajuste (overfitting). El sobreajuste ocurre cuando el modelo ha aprendido los datos de entrenamiento de manera demasiado específica, incluyendo el ruido y las particularidades de ese conjunto. Es como si el modelo memorizara las respuestas en lugar de entender los conceptos. Como resultado, el modelo tiene un rendimiento muy bueno en los datos que ya ha visto (entrenamiento), pero su capacidad para generalizar datos nuevos y no vistos es deficiente, lo que se refleja en un R2 mucho menor en el conjunto de prueba.

**Proponga una solución basada en lo aprendido en la Fase 1 del taller.**

La Fase 1 se enfocó en el uso del parámetro lambda ( $\lambda$ ) en la regularización, por lo tanto, la solución esta enfoca en corregir el sobreajuste. Se prevé Aumentar el valor de lambda o alpha en MLPRegressor:

En la regresión utilizada en MLPRegressor mediante el parámetro  $\alpha$ ,  $\lambda$  es un término de penalización que se añade a la función de pérdida del modelo. Esta penalización disuade al modelo de tener coeficientes o pesos, en el caso de un MLP, muy grandes. Los coeficientes grandes regularmente indican que el modelo está dando demasiada importancia a ciertos datos de entrenamiento, lo que puede llevar al sobreajuste.

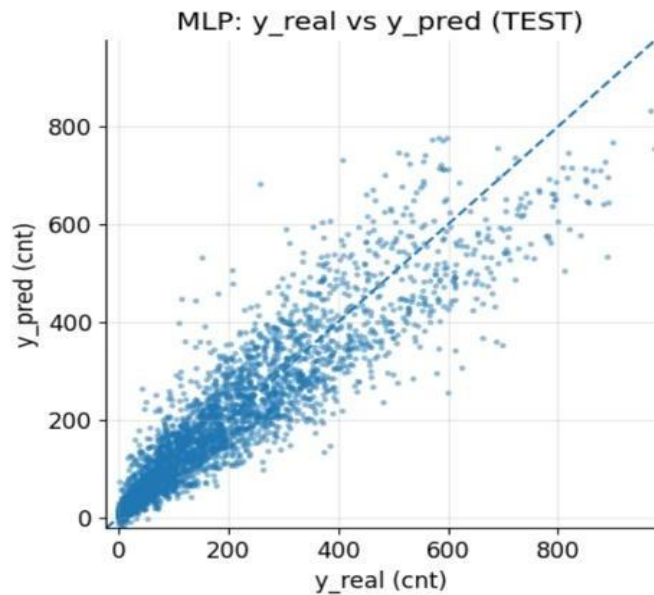
### **Impacto en la complejidad del modelo (Sesgo vs. Varianza):**

Cuando  $\lambda \rightarrow 0$ , el modelo se vuelve más complejo y busca ajustar cada punto de entrenamiento a la perfección, lo que resulta en bajo sesgo, pero alta varianza (sobreajuste). En este escenario, el modelo se ajusta demasiado a los datos de entrenamiento y no generaliza bien.

Cuando aumentamos  $\lambda$ , la penalización se hace más fuerte. Esto empuja a los coeficientes a ser más pequeños. Un modelo con coeficientes más pequeños es intrínsecamente más simple y menos capaz de aprender el ruido específico del conjunto de entrenamiento. Esto aumenta ligeramente el sesgo, pero reduce significativamente la varianza, lo que mejora la capacidad de generalización del modelo en el conjunto de prueba y, por lo tanto, reduce el sobreajuste.

En resumen, al aumentar  $\lambda$ , estamos forzando al modelo a ser más simple y a confiar menos en las particularidades de los datos de entrenamiento, lo que lo hace más robusto y capaz de predecir mejor en datos no vistos.

- **Interpretación de Residuos:** Al observar la gráfica de  $y_{\text{real}}$  vs  $y_{\text{pred}}$ , ¿se identifica algún patrón de error sistemático (ej. subestimación en ventas altas)?



Grafica  $y_{\text{real}}$  vs  $y_{\text{pred}}$  en el modelo MLP sin GridSearch

Al analizar la gráfica de  $y_{\text{real}}$  vs  $y_{\text{pred}}$  para el conjunto de prueba, se observa un patrón de error sistemático, lo que indica que el modelo tiende a subestimar las ventas en los rangos de demanda más altos.

Los puntos de predicción tienden a caer por debajo de la línea diagonal (que representa la predicción perfecta  $y_{\text{real}} = y_{\text{pred}}$ ) cuando los valores reales de cnt son elevados. Esto indica que el modelo no logra capturar la magnitud total de los picos de demanda, prediciendo valores inferiores a los que realmente se observan en esas situaciones.

Adicionalmente, se aprecia una mayor dispersión de los puntos alrededor de la línea diagonal a medida que aumentan los valores de  $y_{\text{real}}$ . Esto sugiere que la incertidumbre y la magnitud del error de predicción son mayores para niveles de demanda más altos, lo que dificulta la estimación precisa de los grandes volúmenes de alquileres.

Este patrón de subestimación en ventas altas es un hallazgo importante que sugiere áreas de mejora para futuros modelos, posiblemente mediante la incorporación de características que capturen mejor los eventos de alta demanda o el uso de modelos que manejen de forma más robusta la heterocedasticidad (variabilidad del error).



## Tarea de Experimentación:

- **Arquitectura (hidden layer sizes):** Compare una red “profunda” (más capas, ej. (32,16,8,4)) contra una “ancha” (una sola capa con muchas neuronas, ej. (64,)).

### Comparación de Arquitecturas MLP: Profunda vs. Ancha

Se realiza la ejecución de los dos modelos para comparar una arquitectura de red neuronal profunda (32, 16, 8, 4) contra una ancha (64,), así se ha podido identificar que la arquitectura profunda fue la de mejor rendimiento.

El rendimiento del modelo “profundo” fue:

|       |                |              |           |
|-------|----------------|--------------|-----------|
| Train | -> MAE: 41.672 | RMSE: 61.806 | R2: 0.885 |
| Test  | -> MAE: 45.196 | RMSE: 68.202 | R2: 0.853 |

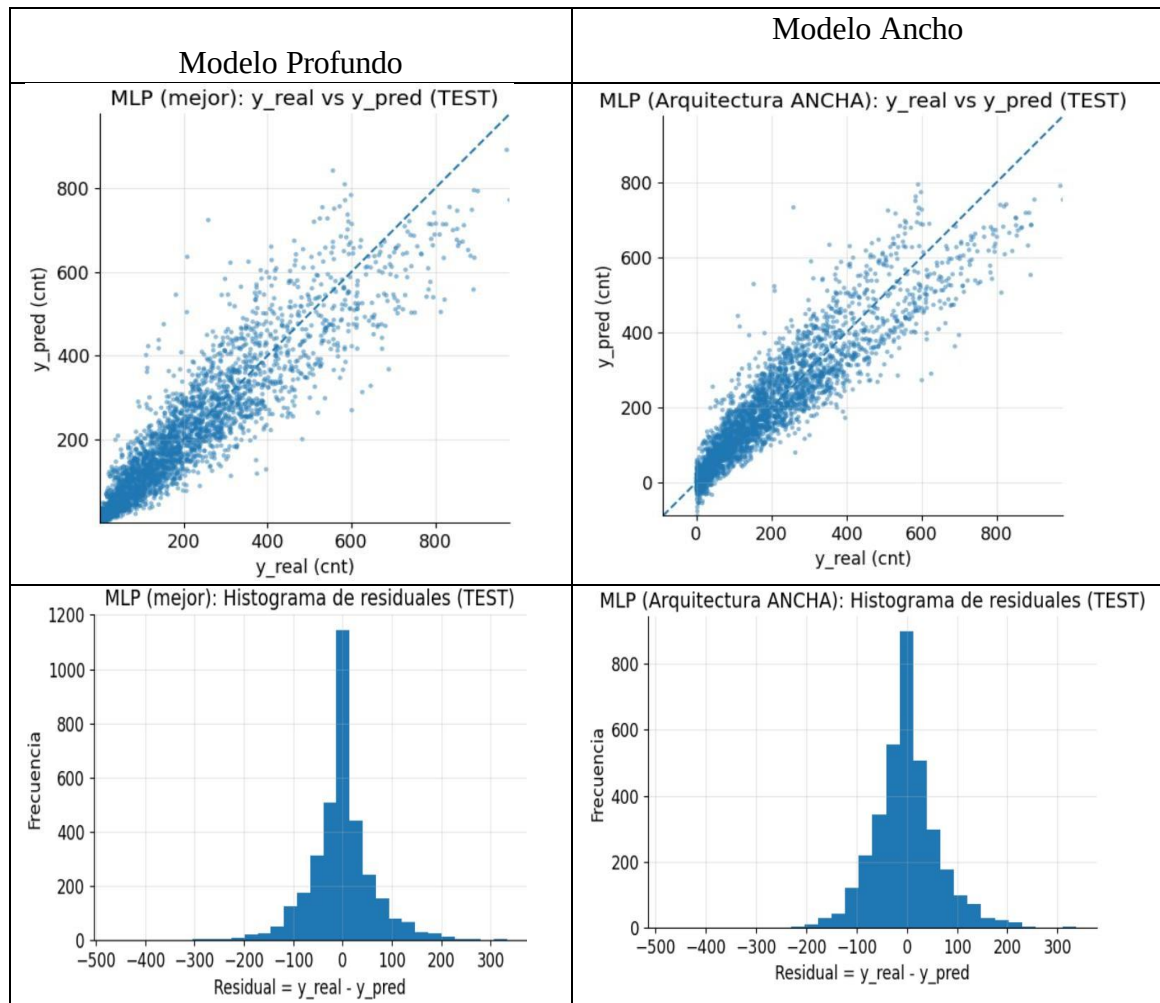
Mientras que el modelo “ancho” arrojó los siguientes resultados:

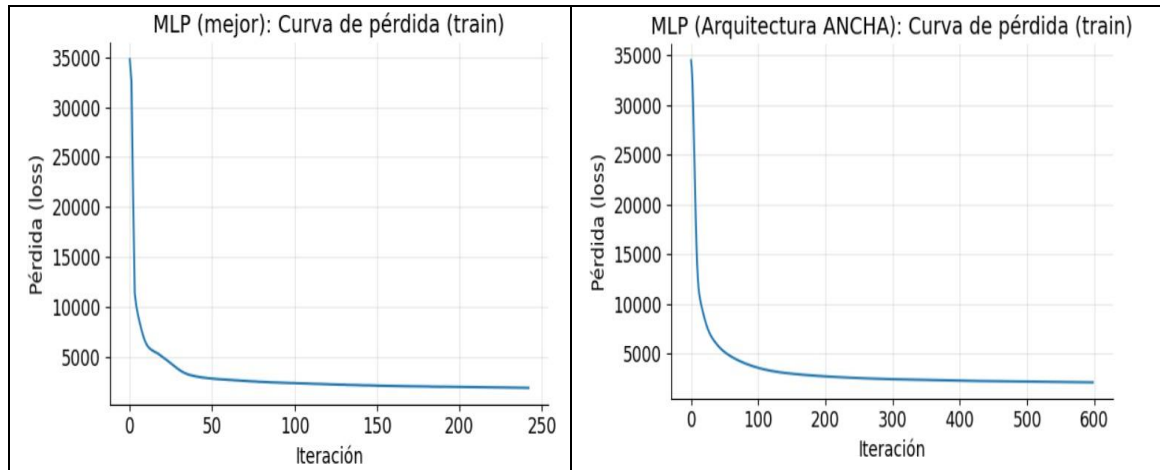
|       |                |              |           |
|-------|----------------|--------------|-----------|
| Train | -> MAE: 46.341 | RMSE: 65.603 | R2: 0.870 |
| Test  | -> MAE: 48.030 | RMSE: 67.574 | R2: 0.856 |

- El modelo profundo mostró un MAE menor que el modelo ancho. Esto indica que, en promedio, la magnitud de los errores absolutos del modelo profundo fue menor.
- El modelo ancho mostró un RMSE ligeramente menor que el modelo profundo en el conjunto de prueba. Esto significa que, en promedio, sus errores de predicción fueron marginalmente más pequeños en términos cuadráticos.
- El modelo profundo mostró un MAE menor que el modelo ancho. Esto indica que, en promedio, la magnitud de los errores absolutos del modelo profundo fue menor.

Para este problema de predicción la red con una arquitectura más profunda, compuesta por capas (32, 16, 8, 4), mostró una mayor capacidad para capturar las complejidades de los datos, resultando en un error de predicción menor en comparación con la red más ancha y simple evaluada.

A continuación, se visualiza el marco diferencial respecto a las gráficas obtenidas:





Estas tres gráficas corresponden al rendimiento del mejor modelo MLP hallado y son clave para visualizar su efectividad.

- **"MLP:  $y_{\text{real}}$  vs  $y_{\text{pred}}$  (TEST)":** Muestra la dispersión de las predicciones del mejor modelo frente a los valores reales en el conjunto de test. Idealmente, los puntos se agruparían cerca de la línea diagonal.
- **"Histograma de residuales (TEST)":** Ilustra la distribución de los errores de predicción. Un modelo ideal tendría residuales centrados alrededor de cero y con una distribución normal.
- **"Curva de pérdida (train)":** Muestra cómo la pérdida del modelo disminuyó durante el entrenamiento. Es útil para entender la convergencia y si el early stopping funcionó correctamente.
- **Regularización (Alpha):** Conecte esto con la Fase 1. Pruebe valores como  $[1e-5, 1e-2, 1.0]$ . ¿Cómo afecta un  $\alpha$  muy elevado a la capacidad de aprendizaje?

```
# 2) Parrilla de hiperparámetros
param_grid = {
    "hidden_layer_sizes": [
        (16, 8, 4), # tu config actual
        (32, 16),   # menos profundo, más ancho
        (32, 16, 8), # un poco mayor capacidad
        (16, 8)     # más simple
    ],
    "alpha": [1e-5, 1e-2, 1.0], | # regularización L2
    "learning_rate_init": [1e-3, 5e-4, 1e-4] # tasa de aprendizaje inicial
```

Los mejores hiperparámetros encontrados incluyen Alpha=0.01.

### Análisis del impacto de un Alpha muy elevado:

El parámetro Alpha en MLPRegressor controla la regularización L2. La regularización añade una penalización a la función de pérdida del modelo en función de la magnitud de sus pesos.

- **Cuando Alpha es muy bajo (cercano a cero):** El modelo tiene más libertad para ajustar sus pesos y puede volverse muy complejo, capturando ruido en los datos de entrenamiento y llevando al **sobreajuste (overfitting)**.

- **Cuando Alpha es muy elevado (como 1.0 en este caso):** La penalización sobre los pesos es muy fuerte. Esto fuerza a los pesos del modelo a ser muy pequeños, lo que simplifica drásticamente el modelo. Si alpha es excesivamente alto, el modelo se vuelve demasiado simple para aprender los patrones subyacentes en los datos, lo que resulta en **infraajuste (underfitting)**.

Alpha=0.01 fue el mejor valor, lo que sugiere que alpha=1.0 llevó a un rendimiento mucho peor (un RMSE de validación cruzada más alto). Un alpha de 1.0 es un valor de regularización muy agresivo que limita severamente la capacidad del modelo para aprender las relaciones complejas de la demanda de bicicletas, forzándolo a una solución demasiado simple. El modelo, en esencia, no pudo "aprender" lo suficiente de los datos debido a la fuerte penalización impuesta a cualquier complejidad en sus pesos.

- **Tasa de Aprendizaje (learning rate init):** Conecte esto con la Fase 2. Analice cómo influye en el número de iteraciones necesarias para que el Early Stopping se active.

```
# 2) Parrilla de hiperparámetros
param_grid = {
    "hidden_layer_sizes": [
        (16, 8, 4), # tu config actual
        (32, 16),   # menos profundo, más ancho
        (32, 16, 8), # un poco mayor capacidad
        (16, 8)     # más simple
    ],
    "alpha": [1e-4, 1e-3, 1e-2], # regularización L2
    "learning_rate_init": [1e-2, 5e-3, 1e-3, 5e-4, 1e-4] # tasa de aprendizaje inicial
}
```

El `learning_rate_init` es fundamental para la eficiencia y el rendimiento del entrenamiento.

Un valor **demasiado alto** puede llevar a una convergencia inestable y Early Stopping prematuro. Un valor **demasiado bajo** ralentiza excesivamente el aprendizaje, impidiendo que el modelo converja a un buen mínimo en un tiempo razonable, y Early Stopping podría no activarse correctamente o detenerse por estancamiento muy leve. El  $LR = 0.01$  fue el que mejor balanceó la velocidad de aprendizaje con la estabilidad, permitiendo a Early Stopping encontrar el punto óptimo para el rendimiento del modelo.

### Preguntas Fase 3B:

- **Consistencia de Parámetros:** Compare el valor óptimo de  $\alpha$  encontrado por el GridSearch con lo que analizó en la sección de Regresión Ridge. ¿Existe una relación entre la complejidad de la red y la necesidad de regularización?

El *GridSearch* encontró como valor óptimo  $\alpha = 0.01$ , lo cual es totalmente consistente con lo analizado en la regresión Ridge. En ambos casos se evidencia que un  $\alpha$  muy bajo genera modelos complejos con riesgo de sobreajuste, y un  $\alpha$  muy alto (ej. 1.0) simplifica excesivamente el modelo, produciendo infraajuste.



Dado que la arquitectura profunda (32,16,8,4) fue la de mejor desempeño, se confirma que, a mayor complejidad de la red, mayor es la necesidad de una regularización adecuada para evitar sobreajuste.

El valor intermedio  $\alpha = 0.01$  logra ese equilibrio entre sesgo y varianza, permitiendo que el modelo aprenda patrones complejos sin perder capacidad de generalización.

- **Early Stopping:** Explique cómo este mecanismo ayuda a optimizar el tiempo de cómputo y a prevenir el sobreajuste basado en sus resultados.

El *early stopping* optimiza tanto el tiempo de cómputo como la generalización del modelo. Detiene el entrenamiento cuando la pérdida deja de mejorar, evitando iteraciones innecesarias. En el desarrollo cuantitativo del taller se observa que la curva de pérdida converge, por lo que continuar entrenando no aporta mejoras.

La prevención de sobreajuste evita que el modelo siga ajustándose al ruido del entrenamiento, deteniéndose en el punto donde el rendimiento es óptimo en validación. Además, el learning rate (0.01) permitió que este mecanismo funcionara correctamente, encontrando el punto óptimo sin convergencia inestable ni estancamiento.

- **Conclusión de Negocio:** Basándose en el mejor modelo encontrado, ¿cuál es el margen de error promedio que la empresa debería esperar al predecir sus ventas?

El mejor modelo es el que presenta menor MAE, lo que indica que el error promedio en las predicciones es bajo en términos absolutos. Sin embargo, existe subestimación en ventas altas y mayor dispersión en esos rangos.

Respecto al margen de error esperado, la empresa debería esperar un error promedio bajo según el MAE, pero con mayor incertidumbre en escenarios de alta demanda.

¿Es el modelo lo suficientemente robusto para decisiones presupuestales? Justifique.

El modelo es parcialmente robusto, con las siguientes consideraciones:

El modelo tiene a su favor un alto desempeño general debido al cálculo de un menor MAE. También cuenta con una correcta generalización gracias a regularización y early stopping, lo que le otorga la capacidad de capturar patrones complejos.

Esto no lo exime de limitaciones, debido a que subestima ventas en picos altos, y mayor error en escenarios críticos de alta demanda.

En conclusión, el modelo es adecuado para decisiones presupuestales generales y estimaciones promedio, pero no es completamente confiable para escenarios de alta demanda, por lo que se recomienda complementarlo con análisis adicionales en esos casos.

Fase 4: Propagación hacia Adelante (Forward Propagation) Manual.

Fase 4:

Valores Parametros  $x: [0,5, 0,3, 0,2]^T$

Capa 1 (Oculta):

$$W^{(1)} = \begin{pmatrix} 0,2 & 0,5 & -0,1 \\ 0,8 & -0,3 & 0,4 \\ -0,5 & 0,1 & 0,2 \end{pmatrix}, b^{(1)} = \begin{pmatrix} 0,1 \\ -0,2 \\ 0,0 \end{pmatrix}$$

Puntuaje de Capa Oculta

$$z^{(1)} = W^{(1)}x + b^{(1)}$$

Neurona 1

$$0,2 \times (0,5) + 0,5 \times (0,3) + (-0,1 \times 0,2) = 0,23$$

$$z^{(1)} = 0,23 + 0,1 = 0,33$$

Neurona 2

$$0,8 \times (0,5) + (-0,3 \times 0,3) + (0,4 \times 0,2) = 0,39$$

$$z^{(2)} = 0,39 + (-0,2) = 0,19$$

Neurona 3

$$(-0,5 \times 0,5) + (0,1 \times 0,3) + (0,2 \times 0,2) = -0,18$$

$$z^{(3)} = -0,18$$

$$z^{(1)} (0,33, 0,19, -0,18)$$

## Activación de la Capa Oculta

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma(0,33) = \frac{1}{1 + e^{-0,33}} = 0,5817$$

$$\sigma(0,19) = \frac{1}{1 + e^{-0,19}} = 0,5473$$

$$\sigma(-0,18) = \frac{1}{1 + e^{0,18}} = 0,4551$$

$$A^{(1)} = (0,5817, 0,5473, 0,4551)$$

Puntuaje Capa de Salida

$$z^{(2)} = W^{(2)} A^{(1)} + b^{(2)}$$

Neurona 1

$$(0,5 \times 0,5817) + (-0,4 \times 0,5473) + (0,1 \times 0,4551) = 0,1174$$

$$z_1^{(2)} = 0,1174 + (-0,1) = 0,0174$$

Neurona 2

$$(0,2 \times 0,5817) + (0,8 \times 0,5473) + (-0,5 \times 0,4551) = 0,3266$$

$$z_2^{(2)} = 0,3266 + 0,2 = 0,5266$$

Neurona 3

$$(-0,1 \times 0,5817) + (0,3 \times 0,5473) + (0,7 \times 0,4551) = 0,4245$$

$$z_{(3)}^{(2)} = 0,4245 + 0,1 = 0,5245$$

$$z^{(2)} = (0,0174, 0,5266, 0,5245)$$

Probabilidad de Clase

$$\text{Softmax}_{(z_i)} = \frac{e^{z_i}}{\sum_{j=1}^3 e^{z_j}}$$

$$e^{0,0174} = 1,0175$$

$$e^{0,5245} = 1,6896$$

$$e^{0,5266} = 1,6931$$

$$\Sigma = 4,4002$$

Probabilidad

$$P_1 = \frac{1,0175}{4,4002} = 0,2312$$

$$P_2 = \frac{1,6896}{4,4002} = 0,3839$$

$$P_3 = \frac{1,6931}{4,4002} = 0,3847$$

$$(P_1, P_2, P_3) \quad 23,12\%, 38,39\%, 38,47\%$$



- **Interpretación de Resultados:** Según las probabilidades obtenidas, ¿en qué categoría clasifica la red a esta campaña publicitaria?

Según las probabilidades obtenidas con Softmax, la red clasifica esta campaña publicitaria en la Categoría 2 (Impacto Medio), ya que es la clase con la mayor probabilidad con un 38.47%. La Categoría 3 le queda muy cerca con un 38.39%, lo que indica que la campaña tiene un desempeño entre medio y alto, pero aún no alcanza un impacto fuerte contundente. La Categoría 1 queda descartada con solo un 23.12%, indicando que la campaña definitivamente no tiene un bajo impacto.

- **Función de Activación:** ¿Por qué utilizamos Softmax en la última capa en lugar de una función lineal o una simple sigmoide? Explique en términos de la suma de las probabilidades.

Utilizamos Softmax porque garantiza que la suma de todas las probabilidades sea exactamente 1, lo cual es fundamental para la clasificación multiclase. Si usáramos una función lineal, los valores de salida podrían ser cualquier número, no estarían acotados entre 0 y 1, y su suma no tendría ningún significado, por lo que no podríamos interpretarlos como probabilidades. Si usáramos una sigmoide simple, cada neurona de salida daría un valor entre 0 y 1 de forma independiente, pero la suma de las tres salidas no sería 1, por lo que podrían darse casos donde las tres clases tengan probabilidades altas al mismo tiempo sin que compitan entre sí, lo cual no tiene sentido cuando queremos que las clases sean mutuamente excluyentes. Softmax resuelve esto porque normaliza todas las salidas entre sí, obligando a que presenten una distribución de probabilidad donde la suma es siempre 1 y cada clase compite con las demás.

- **Rol del Bias:** ¿Qué sucederá con la capacidad de representación de la red si todos los vectores de sesgo (b) fueran cero?

Si todos los vectores de sesgo fueran cero, la red perdería la capacidad de desplazar el punto de activación de cada neurona, lo que significa que todas las neuronas estarían obligadas a activarse únicamente en torno al origen (valor 0). El bias actúa como un umbral que permite a la neurona decidir a partir de qué nivel de entrada comienza a activarse, y sin él, la red solo podría aprender patrones que pasen exactamente por cero, reduciendo drásticamente su capacidad para representar relaciones complejas en los datos. En términos prácticos, las sumatorias  $Z^{(1)}$  y  $Z^{(2)}$  cambiarían porque perderían los términos de sesgo, por ejemplo la neurona 1 de la capa oculta pasaría de 0.33 a 0.23,



la neurona 2 de 0.19 a 0.39, y la neurona 3 se mantendría en -0.18 porque su bias ya era 0. Esto alteraría todas las activaciones posteriores y el resultado final de la clasificación, demostrando que cada bias cumple un rol específico en el ajuste fino de cada neurona.

### Fase 5: Evaluación de Modelos y Toma de Decisiones (Caso: Mantenimiento Predictivo)

- **Análisis de Pesos:** Si el modelo indica que los errores en la Clase 1 (Fallo) pesarán aproximadamente 14.7 veces más que en la Clase 0, ¿qué intenta corregir el algoritmo al configurar el parámetro class weight durante el entrenamiento?

El algoritmo busca corregir el desbalance entre clases, donde hay muchas más muestras de “No Fallo” que de “Fallo”, lo que puede llevar al modelo a predecir siempre la clase mayoritaria sin aprender realmente. Para solucionar esto, se usa `class_weight='balanced'`, asignando mayor peso a la clase minoritaria (Fallo), proporcionalmente a su escasez. Esto hace que los errores en la clase de fallos sean mucho más penalizados durante el entrenamiento, obligando al modelo a prestarles atención. De esta forma, el modelo equilibra la importancia de ambas clases y mejora su capacidad de detección. En contextos industriales, esto es crítico, ya que no detectar un fallo es mucho más costoso que generar una falsa alarma.

- **La Trampa de la Exactitud:** Explique porque en este caso un Accuracy del 95% podría considerarse un resultado deficiente para la gestión de mantenimiento si el modelo no logra detectar los fallos reales.

Un Accuracy del 95% podría ser engañoso porque, si el 94% de tus datos corresponden a equipos sin fallo, el modelo podría simplemente predecir "No Falló" para todos los equipos y alcanzaría ese 95% sin haber aprendido absolutamente nada sobre cómo identificar un fallo real. En mantenimiento industrial, lo que realmente importa es detectar los fallos a tiempo para evitar averías costosas, accidentes o paradas de producción, y no acertar en la mayoría de casos que ya funcionan bien. Si el modelo clasifica todos los fallos reales como "No Falló", el coste de no detectarlos puede ser muchísimo mayor que el beneficio de acertar en los equipos sanos, ya que un fallo no detectado puede derivar en reparaciones caras, pérdida de producción o incluso riesgos para la seguridad de los operarios. Por eso, en estos casos es mucho más importante analizar métricas como el recall de la Clase 1, que mide cuántos fallos reales está

detectando el modelo, más que quedarse con una exactitud general que oculta que el modelo está fallando justo donde más duele.

• Interpretación de la Matriz de Confusión:

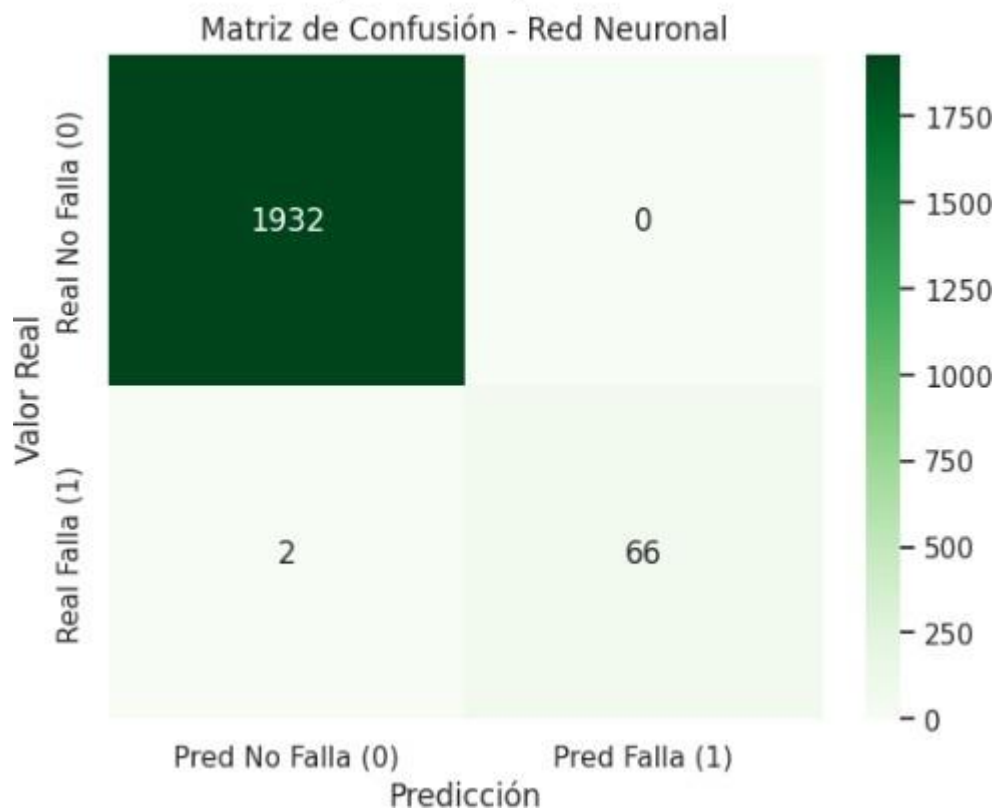


Figura. Matriz de confusión obtenida por el método Red neuronal en el modelo de mantenimiento predictivo.

• Métricas calculadas y respuestas:

1. Precisión:  $\frac{TP}{TP + FP} = \frac{1932}{1932 + 0} = 1$

2. Recall (Sensibilidad):  
 $\frac{TP}{TP + FN} = \frac{1932}{1932 + 2} = 0.9989$   
 P., El modelo logra captar el 99,89% de los fallos

3. Costo total del Error  
 $Costo = (FN \times 5000) + (FP \times 200)$   
 $Costo = (3 \times 5000) + (0 \times 200)$   
 $Costo = \$15.000$

Preguntas de Reflexión y Negocio Fase 5

- **Trade-off entre Recall y Precisión:** En el contexto de evitar paradas de línea crítica, ¿preferiría usted un modelo con un Recall del 98% pero una Precisión del 60%, o viceversa? Justifique su respuesta basándose en los costos calculados anteriormente.

En este caso preferiría un modelo con Recall del 98% y Precisión del 60%, porque el costo de no detectar un fallo real (Falso Negativo) es de \$5,000, mientras que el costo de una falsa alarma (Falso Positivo) es solo de \$200, es decir, un fallo no detectado cuesta 25 veces más que una alerta innecesaria. Si el modelo tiene solo un 60% de Recall, estaría dejando pasar el 40% de los fallos reales, lo que se traduciría en paradas de línea críticas y costos de reparación altísimos que podrían haberse evitado. Con un Recall del 98%, el modelo fallaría en detectar apenas el 2% de los fallos, y aunque el 40% de las alertas sean falsas, el costo de esas falsas alarmas es mucho menor y se compensa con la tranquilidad de que casi ningún fallo real se pasa por alto. En mantenimiento industrial, es mejor tener

que revisar un equipo que estaba bien que quedarse sin revisar uno que estaba a punto de fallar y causar una parada de producción costosa.

- **Curva de Aprendizaje:** Observe la gráfica de loss (pérdida) y accuracy durante el entrenamiento. ¿Considera que 30 épocas son suficientes para que el modelo converja? ¿Qué sucede con el error de validación comparado con el de entrenamiento?

Sí, 30 épocas son más que suficientes porque el modelo prácticamente converge desde la época 5, donde el Recall de entrenamiento llega al 96.7% y la accuracy al 99.5%, y a partir de ahí los valores se estancan sin mejorar significativamente, por lo que hubieras podido detener el entrenamiento en la época 8 o 10 con el mismo resultado. Sobre el error de validación comparado con el de entrenamiento, algo muy interesante es que el val\_loss es consistentemente más bajo que el loss de entrenamiento, bajando de 0.53 hasta 0.02, lo cual indica que no hay overfitting y que el modelo generaliza bien a datos que no ha visto durante el entrenamiento. Esta diferencia a favor de la validación se puede deber a que el conjunto de validación es pequeño y posiblemente más fácil de clasificar, lo que produce esas métricas perfectas de 100% que deben tomarse con cautela al momento de evaluar el rendimiento real en producción.

- **Conclusión Final del Taller:** Tras recorrer desde la regresión analítica hasta la clasificación con redes neuronales, ¿cómo puede el uso de métricas avanzadas (como el Recall) transformar la rentabilidad de una empresa de manufactura?

El uso de métricas avanzadas transforma la rentabilidad de una empresa de manufactura porque permite dejar de evaluar los modelos por resultados superficiales y empezar a tomar decisiones basadas en el impacto económico real de cada predicción. El porcentaje de variable explicable ( $R^2$ ) te dice qué tan confiable es tu modelo para predecir el desgaste de los equipos, mientras que el error promedio en unidades reales te traduce ese error a algo que el ingeniero de mantenimiento puede entender, por ejemplo saber que el modelo se equivoca en promedio por solo 2 horas en la estimación del tiempo restante de vida de una máquina. La penalización de errores al cuadrado asegura que los errores grandes se castiguen fuertemente, evitando que el modelo cometa predicciones que podrían llevar a que una máquina se rompa sin previo aviso y cause una parada de producción costosa.

Y finalmente, en el contexto de clasificación, el Recall es la métrica que directamente protege la rentabilidad porque mide cuántos fallos reales está detectando el modelo, y un fallo no detectado puede costar miles de dólares en reparaciones, paradas de línea y pérdida de clientes, mientras que una falsa alarma solo cuesta una revisión innecesaria que es mucho más barata. Al combinar todas estas métricas con la penalización económica de errores que vimos en los costos de Falsos Negativos y Falsos Positivos, la empresa puede optimizar su mantenimiento predictivo de forma que cada decisión tomada por el modelo se traduzca en ahorros reales, menos paradas no planificadas y una mayor vida útil de los equipos, lo cual impacta directamente en la rentabilidad del negocio.